

Python para Backend

Componentes en

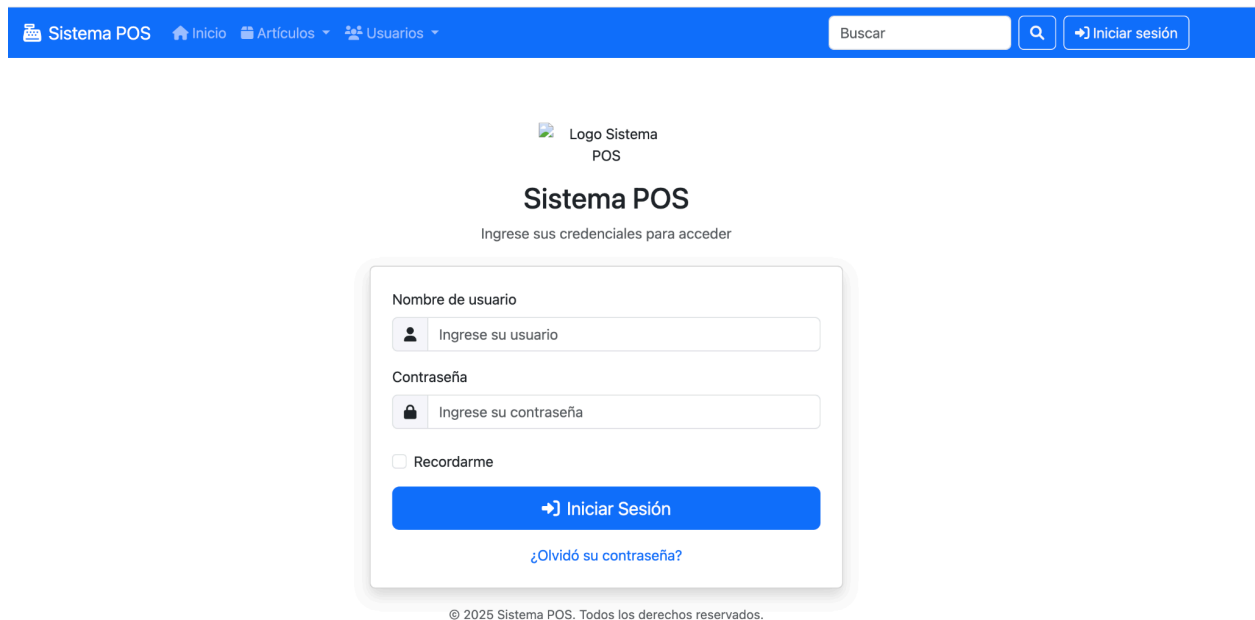
Django

Javier Villegas

28 de abril 2025

Introducción

Para esta sesión vamos a crear un mini aplicativo web para un punto de venta, para ello utilizaremos el modelo presentado en la sesión 3, nuestro website debe quedar así:



Sistema POS

Inicio Artículos Usuarios

Buscar Iniciar sesión

Logo Sistema POS

Sistema POS

Ingrese sus credenciales para acceder

Nombre de usuario

Ingrese su usuario

Contraseña

Ingrese su contraseña

☐ Recordarme

Iniciar Sesión

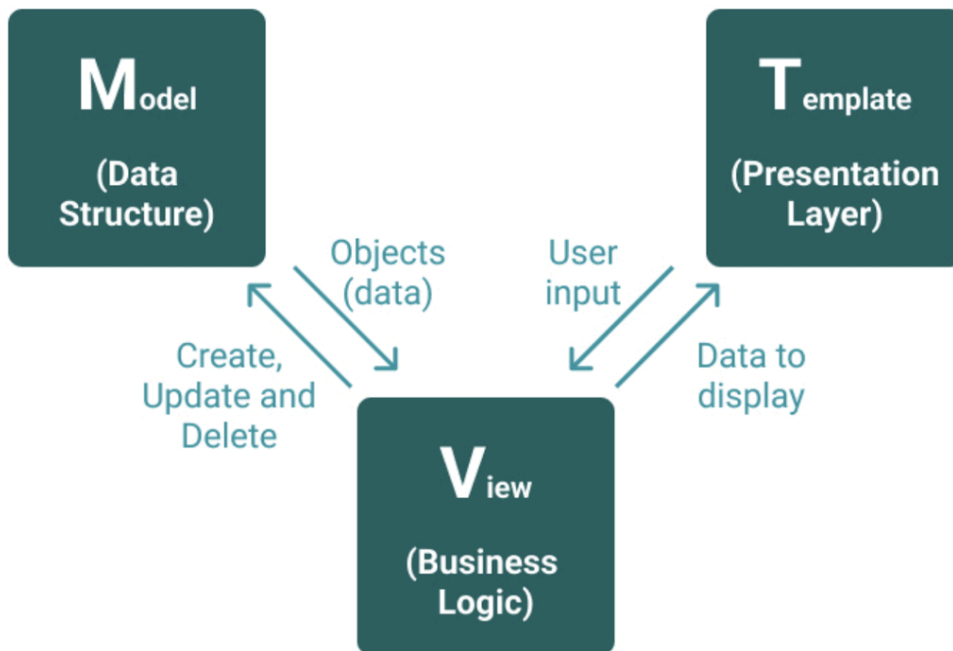
[¿Olvidó su contraseña?](#)

© 2025 Sistema POS. Todos los derechos reservados.

En esta sesión introductoria vamos a ver los siguientes temas:

1. Templates en Django
2. Archivos Estáticos
3. Vistas
4. Formularios

El modelo MVT (Model-View-Template) en Django



Django se basa en la arquitectura MVT (Model-View-Template). MVT es un patrón de diseño de software para desarrollar una aplicación web.

La estructura MVT tiene las siguientes tres partes -

- **Modelo:** El modelo va a actuar como la interfaz de tus datos. Es responsable de mantener los datos. Es la estructura lógica de datos detrás de toda la aplicación y está representada por una base de datos (generalmente bases de datos relacionales como MySQL, Postgres).
- **View:** La Vista es la interfaz de usuario - lo que ves en tu navegador cuando renderizas un sitio web. Está representada por archivos HTML/CSS/Javascript y Jinja. Para saber más
- **Template:** Una plantilla consiste en partes estáticas de la salida HTML deseada, así como alguna sintaxis especial que describe cómo se insertará el contenido dinámico.

Django Templates

Las plantillas son la tercera y más importante parte de la estructura MVT de Django. Una plantilla en Django está básicamente escrita en HTML, CSS y Javascript en un archivo .html. El framework Django maneja y genera eficientemente páginas web HTML dinámicas que son visibles para el usuario final. Django funciona principalmente con un backend por lo que, con el fin de proporcionar un frontend y proporcionar un diseño a nuestro sitio web, utilizamos plantillas. Hay dos métodos para añadir la plantilla a nuestro sitio web dependiendo de nuestras necesidades.

Podemos utilizar un único directorio de plantillas que se extenderá por todo el proyecto. Para cada aplicación de nuestro proyecto, podemos crear un directorio de plantillas diferente.

Para nuestro proyecto actual, crearemos un único directorio de plantillas que se extenderá a lo largo de todo el proyecto por simplicidad. Las plantillas a nivel de aplicación se utilizan generalmente en proyectos grandes o en caso de que queramos proporcionar un diseño diferente a cada componente de nuestra página web.

Paso 1: Configurar la estructura de templates

Para ello, vamos a ir al archivo `pos_project/settings.py` y ubicamos la sección `TEMPLATES`

Python

```
TEMPLATES = [  
    {  
        # Template backend to be used, For example Jinja  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        ## Path definition of templates folder .  
        'DIRS': [BASE_DIR/'templates'],  
        'APP_DIRS': True,  
        # options to configure  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',
```

```
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
    ],
},
},
]
```

Paso 2: Crear la estructura de directorios

Vamos a crear la siguiente estructura en nuestro proyecto. Además, de las carpetas debemos crear los archivos.

Para descargar los archivos estáticos a emplear en esta sesión, hacemos clic en el siguiente enlace:

[static files](#)

```
pos_project/
├── templates/
│   ├── base.html
│   ├── navbar.html
│   ├── accounts/
│   │   ├── login.html
│   │   └── profile.html
│   ├── core/
│   │   ├── index.html
│   │   ├── articulos/
│   │   │   ├── list.html
│   │   │   └── detail.html
│   │   └── grupos/
│   │       ├── list.html
│   │       └── detail.html
├── static/
│   ├── css/
│   │   └── styles.css
│   ├── js/
│   │   └── main.js
│   └── img/
│       └── logo.png
```

Paso 3: Configurar archivos estáticos en settings.py

Los archivos estáticos son aquellos que no cambian, como imágenes, hojas de estilo (CSS), archivos JavaScript, fuentes, etc. Django proporciona una forma de gestionar y servir estos archivos para su uso en el front-end de la aplicación. Los archivos estáticos usualmente son colocados en la carpeta static. Esta carpeta tiene subcarpetas como css, images o js. Este directorio debe ser registrado en el archivo settings.py.

Para habilitar el manejo de archivos estáticos en Django, debes asegurarte de tener configuradas las siguientes opciones en el archivo `pos_project/settings.py` y agregamos lo siguiente:

Python

```
STATIC_URL = 'static/'

STATICFILES_DIRS = [

    BASE_DIR / "static",

]

STATIC_ROOT = BASE_DIR / "staticfiles"

MEDIA_URL = '/media/'

MEDIA_ROOT = BASE_DIR / 'media'
```

Paso 4: Configurar URLs para servir archivos estáticos en desarrollo

Para ello vamos al archivo `pos_project/urls.py` y configuramos lo siguiente:

Python

```
from django.contrib import admin

from django.urls import path, include

from django.conf import settings

from django.conf.urls.static import static

from django.views.generic import TemplateView


urlpatterns = [

    path('admin/', admin.site.urls),

    path('', TemplateView.as_view(template_name='core/index.html'),
         name='home'),

    # Aquí agregaríamos las URLs de nuestras apps

]
```

```
if settings.DEBUG:

    urlpatterns += static(settings.STATIC_URL,
document_root=settings.STATIC_ROOT)

    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

Paso 5: Crear la plantilla base (base.html)

Ahora vamos a crear la plantilla base de la cual todas las plantillas de nuestro proyecto heredarán.

Python

```
<!DOCTYPE html>

<html lang="es">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block title %}Sistema POS{% endblock %}</title>

    <!-- Bootstrap CSS -->

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    <!-- Font Awesome -->
```



```

    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css
">

    <!-- Estilos personalizados -->

    {% load static %}

    <link rel="stylesheet" href="{% static 'css/styles.css' %}">

    {% block extra_css %}{% endblock %}
</head>
<body>

    <!-- Navbar -->

    {% include 'navbar.html' %}

    <!-- Contenido principal -->

    <main class="container py-4">

        {% if messages %}

        <div class="messages">

            {% for message in messages %}

                <div class="alert alert-{{ message.tags }} alert-dismissible fade
show" role="alert">

                    {{ message }}

                    <button type="button" class="btn-close" data-bs-dismiss="alert"
aria-label="Close"></button>

                </div>

            {% endfor %}

        </div>

    </div>

```

```

        {% endif %}

        {% block content %}{% endblock %}
    </main>

    <!-- Footer -->
    <footer class="bg-dark text-white py-4 mt-5">
        <div class="container">
            <div class="row">
                <div class="col-md-6">
                    <h5>Sistema POS</h5>
                    <p>Una solución completa para gestionar tu negocio</p>
                </div>
                <div class="col-md-6 text-md-end">
                    <p>&copy; {< now "Y" %} Todos los derechos reservados</p>
                </div>
            </div>
        </div>
    </footer>

    <!-- Bootstrap JS -->

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.
js"></script>

    <!-- JavaScript personalizado -->

```

```

<script src="{% static 'js/main.js' %}"></script>

{% block extra_js %}{% endblock %}

</body>

</html>

```

A continuación vamos a comentar algunas líneas que se ven en nuestro archivo base.html

{% static %} tag

template file

```

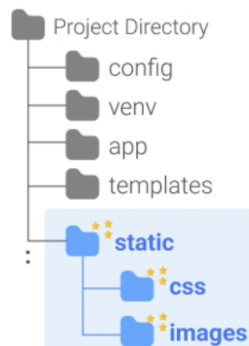
{% load static %}

<link rel='stylesheet' href="{% static 'css/xxx.css' %}">



```

Typical directory structure



settings.py

Defines static directory location

```
STATICFILES_DIRS = [BASE_DIR / 'static']
```

```
{% load static %}
```

- Add this tag in the beginning of each template file

```
{% static 'file path' %}
```

- Define the location of static files
- The **file path** should be written as a relative path from the static directory

En tus plantillas Django, utiliza la etiqueta de plantilla {% static %} para enlazar con tus archivos estáticos. Comienza cargando la biblioteca de etiquetas de plantillas estáticas en la parte superior de tu archivo de plantilla:

Python

```
{% load static %}
```

{% include <file.html> %} tag

La etiqueta include le permite incluir una plantilla dentro de la plantilla actual. Esto es útil cuando se tiene un bloque de contenido que es el mismo para muchas páginas.

Python

```
{% include 'navbar.html' %}
```

Template variables

En las plantillas de Django, puedes renderizar variables poniéndolas dentro de corchetes `{{ }}`:

```
<!-- Contenido principal -->
<main class="container py-4">
    {% if messages %}
    <div class="messages">
        {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert
        </div>
        {% endfor %}
    </div>
    {% endif %}

    {% block content %}{% endblock %}
</main>
```

Template tags

En las plantillas de Django, puedes realizar lógica de programación como ejecutar sentencias if y bucles for.

Estas palabras clave, if y for, se llaman "etiquetas de plantilla" en Django.

Para ejecutar etiquetas de plantilla, las encerramos entre corchetes `{% %}`.

```

<!-- Contenido principal -->
<main class="container py-4">
    {% if messages %}
    <div class="messages">
        {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade show"
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert" aria
        </div>
        {% endfor %}
    </div>
    {% endif %}

    {% block content %}{% endblock %}
</main>

```

Paso 6: Crear el navbar (navbar.html)

Python

```

<nav class="navbar navbar-expand-lg navbar-dark bg-primary">

    <div class="container">

        <a class="navbar-brand" href="{% url 'home' %}">

            <i class="fas fa-cash-register me-2"></i>Sistema POS

        </a>

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav"

            aria-controls="navbarNav" aria-expanded="false"
aria-label="Toggle navigation">

            <span class="navbar-toggler-icon"></span>

        </button>

        <div class="collapse navbar-collapse" id="navbarNav">

```

```

        <ul class="navbar-nav me-auto">

            <li class="nav-item">

                <a class="nav-link {% if request.path == '/' %}active{%
endif %}" href="{% url 'home' %}">

                    <i class="fas fa-home me-1"></i>Inicio

                </a>

            </li>

            <!-- Dropdown para Artículos -->

            <li class="nav-item dropdown">

                <a class="nav-link dropdown-toggle" href="#"
id="articulosDropdown" role="button"

                    data-bs-toggle="dropdown" aria-expanded="false">

                        <i class="fas fa-box me-1"></i>Artículos

                    </a>

                    <ul class="dropdown-menu"
aria-labelledby="articulosDropdown">

                        <li><a class="dropdown-item" href="#">Lista de
Artículos</a></li>

                        <li><a class="dropdown-item" href="#">Nuevo
Artículo</a></li>

                        <li><hr class="dropdown-divider"></li>

                        <li><a class="dropdown-item" href="#">Grupos de
Artículos</a></li>

                        <li><a class="dropdown-item" href="#">Líneas de
Artículos</a></li>

                    </ul>

                </li>

```

```

        <!-- Dropdown para Usuarios -->

        <li class="nav-item dropdown">

            <a class="nav-link dropdown-toggle" href="#"
id="usuariosDropdown" role="button"

                data-bs-toggle="dropdown" aria-expanded="false">

                <i class="fas fa-users me-1"></i>Usuarios

            </a>

            <ul class="dropdown-menu"
aria-labelledby="usuariosDropdown">

                <li><a class="dropdown-item" href="#">Lista de
Usuarios</a></li>

                <li><a class="dropdown-item" href="#">Nuevo
Usuario</a></li>

                <li><hr class="dropdown-divider"></li>

                <li><a class="dropdown-item" href="#">Perfiles</a></li>

            </ul>

        </li>

    </ul>

    <!-- Usuario y Búsqueda -->

    <div class="d-flex">

        <form class="d-flex me-2">

            <input class="form-control me-2" type="search"
placeholder="Buscar" aria-label="Buscar">

            <button class="btn btn-outline-light" type="submit"><i
class="fas fa-search"></i></button>

        </form>

```

```

        {% if user.is_authenticated %}

        <div class="dropdown">

            <button class="btn btn-outline-light dropdown-toggle"
type="button" id="userDropdown"

                data-bs-toggle="dropdown" aria-expanded="false">

                <i class="fas fa-user me-1"></i>{{
user.full_name|truncatechars:15 }}

            </button>

            <ul class="dropdown-menu dropdown-menu-end"
aria-labelledby="userDropdown">

                <li><a class="dropdown-item" href="#"><i class="fas
fa-user-circle me-2"></i>Perfil</a></li>

                <li><a class="dropdown-item" href="#"><i class="fas
fa-cog me-2"></i>Configuración</a></li>

                <li><hr class="dropdown-divider"></li>

                <li><a class="dropdown-item" href="#"><i class="fas
fa-sign-out-alt me-2"></i>Cerrar sesión</a></li>

            </ul>

        </div>

        {% else %}

        <a href="#" class="btn btn-outline-light"><i class="fas
fa-sign-in-alt me-1"></i>Iniciar sesión</a>

        {% endif %}

    </div>

</div>

</div>

</nav>

```


Paso 7: Crear la página de inicio (index.html)

Python

```
{% extends 'base.html' %}

{% block title %}Inicio - Sistema POS{% endblock %}

{% block content %}

<div class="row mb-4">

    <div class="col-md-12">

        <div class="card shadow">

            <div class="card-body">

                <h2 class="card-title text-primary">

                    <i class="fas fa-store me-2"></i>Bienvenido al Sistema POS

                </h2>

                <p class="lead">Sistema integrado para gestión de ventas e
inventario.</p>

                <hr>

                <p>Este sistema le permite administrar sus productos, ventas,
usuarios y reportes de manera eficiente.</p>

            </div>

        </div>

    </div>

</div>

<!-- Tarjetas de estadísticas -->

<div class="row mb-4">
```

```

<div class="col-md-3">
  <div class="card bg-primary text-white shadow-sm mb-3">
    <div class="card-body">
      <div class="d-flex justify-content-between align-items-center">
        <div>
          <h6 class="card-title">Artículos</h6>
          <h2 class="mb-0">{{ total_articulos|default:"0" }}</h2>
        </div>
        <i class="fas fa-box fa-2x"></i>
      </div>
    </div>
    <div class="card-footer bg-primary-dark d-flex justify-content-between align-items-center py-1">
      <small>Ver detalles</small>
      <i class="fas fa-arrow-circle-right"></i>
    </div>
  </div>
</div>

<div class="col-md-3">
  <div class="card bg-success text-white shadow-sm mb-3">
    <div class="card-body">
      <div class="d-flex justify-content-between align-items-center">
        <div>
          <h6 class="card-title">Ventas Hoy</h6>
          <h2 class="mb-0">{{ ventas_hoy|default:"0" }}</h2>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        </div>

        <i class="fas fa-shopping-cart fa-2x"></i>

    </div>

</div>

<div class="card-footer bg-success-dark d-flex
justify-content-between align-items-center py-1">

    <small>Ver detalles</small>

    <i class="fas fa-arrow-circle-right"></i>

</div>

</div>

</div>

<div class="col-md-3">

    <div class="card bg-warning text-white shadow-sm mb-3">

        <div class="card-body">

            <div class="d-flex justify-content-between align-items-center">

                <div>

                    <h6 class="card-title">Usuarios</h6>

                    <h2 class="mb-0">{{ total_usuarios|default:"0" }}</h2>

                </div>

                <i class="fas fa-users fa-2x"></i>

            </div>

        </div>

        <div class="card-footer bg-warning-dark d-flex
justify-content-between align-items-center py-1">

            <small>Ver detalles</small>

```

```

        <i class="fas fa-arrow-circle-right"></i>

    </div>

</div>

</div>

<div class="col-md-3">

    <div class="card bg-danger text-white shadow-sm mb-3">

        <div class="card-body">

            <div class="d-flex justify-content-between align-items-center">

                <div>

                    <h6 class="card-title">Bajo Stock</h6>

                    <h2 class="mb-0">{{ bajo_stock|default:"0" }}</h2>

                </div>

                <i class="fas fa-exclamation-triangle fa-2x"></i>

            </div>

        </div>

        <div class="card-footer bg-danger-dark d-flex justify-content-between align-items-center py-1">

            <small>Ver detalles</small>

            <i class="fas fa-arrow-circle-right"></i>

        </div>

    </div>

</div>

</div>

<!-- Widgets de accesos rápidos -->

```

```

<div class="row">
  <div class="col-md-8">
    <div class="card shadow">
      <div class="card-header bg-white">
        <h5 class="card-title mb-0">
          <i class="fas fa-tasks me-2"></i>Acciones Rápidas
        </h5>
      </div>
      <div class="card-body">
        <div class="row">
          <div class="col-md-3">
            <a href="#" class="btn btn-primary btn-block p-3 mb-3
text-center">
              <i class="fas fa-shopping-cart fa-2x mb-2"></i><br>
              Nueva Venta
            </a>
          </div>
          <div class="col-md-3">
            <a href="#" class="btn btn-success btn-block p-3 mb-3
text-center">
              <i class="fas fa-plus-circle fa-2x mb-2"></i><br>
              Nuevo Artículo
            </a>
          </div>
          <div class="col-md-3">

```

```

text-center">
        <a href="#" class="btn btn-info btn-block p-3 mb-3">
            <i class="fas fa-user-plus fa-2x mb-2"></i><br>
            Nuevo Usuario
        </a>
    </div>
    <div class="col-md-3">
        <a href="#" class="btn btn-warning btn-block p-3 mb-3">
            <i class="fas fa-chart-bar fa-2x mb-2"></i><br>
            Reportes
        </a>
    </div>
</div>
</div>
</div>
</div>
</div>

<div class="col-md-4">
    <div class="card shadow">
        <div class="card-header bg-white">
            <h5 class="card-title mb-0">
                <i class="fas fa-bell me-2"></i>Notificaciones
            </h5>
        </div>
        <div class="card-body p-0">

```

```

<ul class="list-group list-group-flush">
  <li class="list-group-item d-flex align-items-center">
    <div class="bg-warning rounded-circle p-2 me-3">
      <i class="fas fa-exclamation-triangle
text-white"></i>
    </div>
    <div>
      <p class="mb-0">5 productos con stock bajo</p>
      <small class="text-muted">Hace 3 horas</small>
    </div>
  </li>
  <li class="list-group-item d-flex align-items-center">
    <div class="bg-success rounded-circle p-2 me-3">
      <i class="fas fa-check text-white"></i>
    </div>
    <div>
      <p class="mb-0">Venta #1234 completada</p>
      <small class="text-muted">Hace 5 horas</small>
    </div>
  </li>
  <li class="list-group-item d-flex align-items-center">
    <div class="bg-primary rounded-circle p-2 me-3">
      <i class="fas fa-user text-white"></i>
    </div>
    <div>
      <p class="mb-0">Nuevo usuario registrado</p>

```

```

        <small class="text-muted">Ayer</small>
      </div>
    </li>
  </ul>
</div>
<div class="card-footer bg-white">
  <a href="#" class="btn btn-sm btn-outline-primary">Ver
todas</a>
</div>
</div>
</div>
{% endblock %}

```

Paso 8: Crear archivo CSS personalizado (styles.css)

En este paso vamos a crear nuestra hoja de estilos, la misma que será usada durante todo este proyecto, para ello vamos al archivo `static/css/syles.css` y agregamos lo siguiente:

```

Unset

/* Estilos personalizados para el Sistema POS */

/* Variables de colores */
:root {
  --primary-dark: #0d47a1;

```



```
--success-dark: #1b5e20;

--warning-dark: #e65100;

--danger-dark: #b71c1c;
}

/* Estilo para el cuerpo de la página */
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f5f5f5;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

main {
    flex: 1;
}

/* Navbar y componentes de navegación */
.navbar-brand {
    font-weight: bold;
    letter-spacing: 0.5px;
}

.nav-link {
```

```
        font-weight: 500;
    }

    /* Cards y contenedores */
    .card {
        border-radius: 0.5rem;
        border: none;
        transition: transform 0.2s, box-shadow 0.2s;
    }

    .card:hover {
        transform: translateY(-3px);
        box-shadow: 0 10px 20px rgba(0, 0, 0, 0.1);
    }

    .card-header {
        font-weight: 600;
        border-bottom: 1px solid rgba(0, 0, 0, 0.1);
    }

    /* Botones */
    .btn {
        border-radius: 0.4rem;
        font-weight: 500;
    }
```

```
.btn-block {
    width: 100%;
}

/* Fondos para los footers de tarjetas */
.bg-primary-dark {
    background-color: var(--primary-dark);
}

.bg-success-dark {
    background-color: var(--success-dark);
}

.bg-warning-dark {
    background-color: var(--warning-dark);
}

.bg-danger-dark {
    background-color: var(--danger-dark);
}

/* Animación de hover para los enlaces */
a {
    transition: all 0.3s ease;
```

```
}

/* Estilos para tablas */
.table {
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
    border-radius: 0.5rem;
    overflow: hidden;
}

.table th {
    background-color: #f8f9fa;
    border-top: none;
}

/* Formularios */
.form-control:focus {
    box-shadow: 0 0 0 0.2rem rgba(13, 110, 253, 0.25);
    border-color: #86b7fe;
}

/* Paginación */
.page-item.active .page-link {
    background-color: var(--primary-dark);
    border-color: var(--primary-dark);
}
```

```
/* Adaptaciones responsivas */
@media (max-width: 768px) {
    .card-body {
        padding: 1rem;
    }
}
```

Paso 9: Crear archivo JavaScript básico (main.js)

Para este paso vamos a crear algunas funciones en JavaScript que vamos a emplear para la manipulación, para ello vamos al archivo `static/js/main.js` y agregamos el siguiente código:

JavaScript

```
// Archivo main.js para el Sistema POS

// Función para mostrar tooltips de Bootstrap
document.addEventListener('DOMContentLoaded', function() {

    // Inicializar todos los tooltips

    var tooltipTriggerList =
    [].slice.call(document.querySelectorAll('[data-bs-toggle="tooltip"]'));

    tooltipTriggerList.map(function (tooltipTriggerEl) {

        return new bootstrap.Tooltip(tooltipTriggerEl);

    });
});
```

```

// Auto-cerrar alertas después de 5 segundos

setTimeout(function() {

    var alerts = document.querySelectorAll('.alert');

    alerts.forEach(function(alert) {

        var bsAlert = new bootstrap.Alert(alert);

        bsAlert.close();

    });

}, 5000);

});

// Función para confirmar eliminación

function confirmarEliminacion(event, mensaje) {

    if (!confirm(mensaje || '¿Está seguro de que desea eliminar este elemento?')) {

        event.preventDefault();

        return false;

    }

    return true;

}

// Función para actualizar el contador de caracteres en textareas

function actualizarContador(textareaId, contadorId, maxLength) {

    const textarea = document.getElementById(textareaId);

    const contador = document.getElementById(contadorId);

    if (textarea && contador) {

```

```
textarea.addEventListener('input', function() {

    const caracteresRestantes = maxLength - this.value.length;

    contador.textContent = caracteresRestantes;

    if (caracteresRestantes < 20) {

        contador.classList.add('text-danger');

    } else {

        contador.classList.remove('text-danger');

    }

});

}

}

// Función para validar formularios

function validarFormulario(formId) {

    const form = document.getElementById(formId);

    if (form) {

        form.addEventListener('submit', function(event) {

            if (!form.checkValidity()) {

                event.preventDefault();

                event.stopPropagation();

            }

            form.classList.add('was-validated');

        });

    }

}
```

```
});  
  
}
```

Paso 10: Crear la vista para Listado de Productos

Para este paso vamos a crear el listado de productos para ello vamos a escribir el siguiente código:

Python

```
{% extends 'base.html' %}  
  
{% block title %}Listado de Artículos - Sistema POS{% endblock %}  
  
{% block content %}  
<div class="d-flex justify-content-between align-items-center mb-4">  
    <h2 class="h3"><i class="fas fa-boxes me-2"></i>Listado de Artículos</h2>  
    <a href="#" class="btn btn-primary">  
        <i class="fas fa-plus me-1"></i>Nuevo Artículo  
    </a>  
</div>  
  
<div class="card shadow">  
    <div class="card-header bg-white d-flex justify-content-between">  
        <h5 class="mb-0">Artículos</h5>
```



```

<div class="input-group" style="width: 300px;">
    <input type="text" class="form-control" placeholder="Buscar
artículo..." id="searchArticulo">
    <button class="btn btn-outline-secondary" type="button">
        <i class="fas fa-search"></i>
    </button>
</div>
</div>

```

```

<div class="card-body p-0">
    <div class="table-responsive">
        <table class="table table-hover mb-0">
            <thead>
                <tr>
                    <th>Código</th>
                    <th>Descripción</th>
                    <th>Grupo</th>
                    <th>Línea</th>
                    <th>Stock</th>
                    <th>Precio</th>
                    <th>Acciones</th>
                </tr>
            </thead>
            <tbody>
                {% for articulo in articulos %}
                <tr>

```

```

<td>{{ articulo.codigo_articulo }}</td>
<td>{{ articulo.descripcion }}</td>
<td>{{ articulo.grupo.nombre_grupo }}</td>
<td>{{ articulo.linea.nombre_linea }}</td>
<td>
    {% if articulo.stock < 10 %}
    <span class="badge bg-danger">{{ articulo.stock
}}</span>

    {% else %}
    <span class="badge bg-success">{{ articulo.stock
}}</span>

    {% endif %}
</td>
<td>${{{ articulo.listaprecio.precio_1 }}}</td>
<td>
    <div class="btn-group btn-group-sm">
        <a href="#" class="btn btn-primary"
data-bs-toggle="tooltip" title="Ver detalle">
            <i class="fas fa-eye"></i>
        </a>
        <a href="#" class="btn btn-warning"
data-bs-toggle="tooltip" title="Editar">
            <i class="fas fa-edit"></i>
        </a>
        <a href="#" class="btn btn-danger"
onclick="confirmarEliminacion(event, '¿Eliminar este artículo?')"
data-bs-toggle="tooltip" title="Eliminar">
            <i class="fas fa-trash"></i>

```

```

        </a>

    </div>

</td>

</tr>

{% empty %}

<tr>

    <td colspan="7" class="text-center py-3">No hay
artículos para mostrar</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

{% if articulos.has_other_pages %}

<div class="card-footer bg-white">

    <nav aria-label="Paginación">

        <ul class="pagination justify-content-center mb-0">

            {% if articulos.has_previous %}

            <li class="page-item">

                <a class="page-link" href="?page={{
articulos.previous_page_number }}">

                    <i class="fas fa-chevron-left"></i>

                </a>

            </li>

```

```

        {% else %}

        <li class="page-item disabled">

            <span class="page-link"><i class="fas
fa-chevron-left"></i></span>

        </li>

        {% endif %}

{% for i in articulos.paginator.page_range %}

    {% if articulos.number == i %}

    <li class="page-item active">

        <span class="page-link">{{ i }}</span>

    </li>

    {% else %}

    <li class="page-item">

        <a class="page-link" href="?page={{ i }}">{{ i }}</a>

    </li>

    {% endif %}

{% endfor %}

{% if articulos.has_next %}

<li class="page-item">

    <a class="page-link" href="?page={{
articulos.next_page_number }}">

        <i class="fas fa-chevron-right"></i>

    </a>

</li>

```

```

        {% else %}

        <li class="page-item disabled">

            <span class="page-link"><i class="fas
fa-chevron-right"></i></span>

        </li>

        {% endif %}

    </ul>

</nav>

</div>

{% endif %}

</div>

{% endblock %}

{% block extra_js %}

<script>

    // Inicializar tooltips

    document.addEventListener('DOMContentLoaded', function() {

        var tooltipTriggerList =
[ ].slice.call(document.querySelectorAll('[data-bs-toggle="tooltip"]'));

        tooltipTriggerList.map(function (tooltipTriggerEl) {

            return new bootstrap.Tooltip(tooltipTriggerEl);

        });

        // Filtro rápido para la tabla

        const searchInput = document.getElementById('searchArticulo');

        if (searchInput) {

```

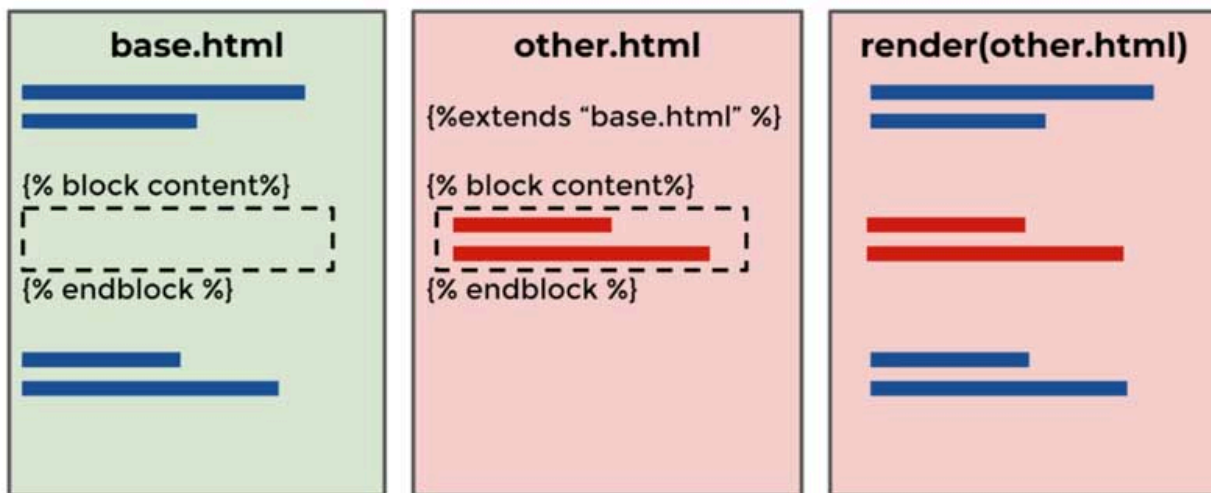
```
searchInput.addEventListener('keyup', function() {  
    const searchTerm = this.value.toLowerCase();  
    const tableRows = document.querySelectorAll('tbody tr');  
  
    tableRows.forEach(row => {  
        const text = row.textContent.toLowerCase();  
        const isMatch = text.includes(searchTerm);  
        row.style.display = isMatch ? '' : 'none';  
    });  
});  
});  
</script>  
{% endblock %}
```

Vamos a comentar algunos etiquetas

{% extends <nombreplantilla.html> %}

En las plantillas de Django, la etiqueta {% extends %} habilita la herencia de plantillas, permitiéndote construir sobre una plantilla base. Esto promueve la reutilización de código y mantiene la coherencia en toda la aplicación web.

Para usar {% extends %}, colócala al principio de tu plantilla hija. Especifica la plantilla padre de la que heredar, utilizando una cadena literal para un nombre de plantilla fijo o una variable para uno dinámico.



`{% block <nombre bloque> %}`

el `{% block loquesea %}` sirve para reemplazar o cambiar una parte de la página o plantilla base principal cuando estás haciendo herencia de plantillas. Es decir si tienes un `index.html` principal donde tienes definido un bloque así `{% block loquesea %} hola {% endblock %}` y luego heredas de esa plantilla en un `ejemplo.html`, para cambiar ese "hola" específicamente de ese bloque, lo reemplazarías utilizando el mismo nombrado de bloque: `{% block loquesea %} nuevo hola {% endblock %}` y el resto de página se mantendría igual.

Paso 11: Crear una vista para el detalle de artículo (detail.html)

Para este paso vamos a crear el detalle de productos para ello vamos a escribir el siguiente código:

Python

```
{% extends 'base.html' %}

{% block title %}{{ articulo.descripcion }} - Sistema POS{% endblock %}
```

```

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3">

        <i class="fas fa-box me-2"></i>Detalle de Artículo

    </h2>

    <div>

        <a href="{% url 'articulos_list' %}" class="btn btn-outline-secondary me-2">

            <i class="fas fa-arrow-left me-1"></i>Volver

        </a>

        <a href="{% url 'articulo_edit' articulo.articulo_id %}" class="btn btn-warning">

            <i class="fas fa-edit me-1"></i>Editar

        </a>

    </div>

</div>

<div class="row">

    <div class="col-md-8">

        <div class="card shadow mb-4">

            <div class="card-header bg-white">

                <h5 class="mb-0">Información del Artículo</h5>

            </div>

            <div class="card-body">

                <div class="row mb-4">

                    <div class="col-md-6">

```



```

        <p class="mb-1 text-muted">Código:</p>
        <h5>{{ articulo.codigo_articulo }}</h5>
    </div>
    <div class="col-md-6">
        <p class="mb-1 text-muted">Código de Barras:</p>
        <h5>{{ articulo.codigo_barras|default:"No especificado"
    }}</h5>

    </div>
</div>

<div class="mb-4">
    <p class="mb-1 text-muted">Descripción:</p>
    <h4>{{ articulo.descripcion }}</h4>
</div>

<div class="row mb-4">
    <div class="col-md-6">
        <p class="mb-1 text-muted">Presentación:</p>
        <h5>{{ articulo.presentacion }}</h5>
    </div>
    <div class="col-md-6">
        <p class="mb-1 text-muted">Stock:</p>
        <h5>
            {% if articulo.stock < 10 %}
            <span class="badge bg-danger">{{ articulo.stock
    }}</span>

```

```

        {% else %}

        <span class="badge bg-success">{{ articulo.stock
    }}</span>

        {% endif %}

    </h5>

</div>

</div>

</div>

<div class="row">

    <div class="col-md-6">

        <p class="mb-1 text-muted">Grupo:</p>

        <h5>{{ articulo.grupo.nombre_grupo }}</h5>

    </div>

    <div class="col-md-6">

        <p class="mb-1 text-muted">Línea:</p>

        <h5>{{ articulo.linea.nombre_linea }}</h5>

    </div>

</div>

</div>

</div>

<!-- Movimientos del artículo (opcional) -->

<div class="card shadow">

    <div class="card-header bg-white">

        <h5 class="mb-0">Últimos Movimientos</h5>

    </div>

```

```

<div class="card-body p-0">
  <div class="table-responsive">
    <table class="table table-hover mb-0">
      <thead>
        <tr>
          <th>Fecha</th>
          <th>Tipo</th>
          <th>Cantidad</th>
          <th>Usuario</th>
        </tr>
      </thead>
      <tbody>
        <!-- Ejemplo de movimientos, en un caso real
vendrían del backend -->
        <tr>
          <td>25/04/2025 15:30</td>
          <td><span class="badge
bg-success">Entrada</span></td>
          <td>+10</td>
          <td>Admin</td>
        </tr>
        <tr>
          <td>24/04/2025 14:20</td>
          <td><span class="badge
bg-danger">Salida</span></td>
          <td>-2</td>

```

```

        <td>Vendedor1</td>

    </tr>

    <tr>

        <td>23/04/2025 09:15</td>

        <td><span class="badge
bg-primary">Ajuste</span></td>

        <td>+5</td>

        <td>Admin</td>

    </tr>

</tbody>

</table>

</div>

</div>

</div>

</div>

<div class="col-md-4">

    <!-- Precios -->

    <div class="card shadow mb-4">

        <div class="card-header bg-white">

            <h5 class="mb-0">Precios</h5>

        </div>

        <div class="card-body">

            <ul class="list-group list-group-flush">

                <li class="list-group-item d-flex justify-content-between
align-items-center">

```

```

        <span>Precio 1:</span>

        <span class="badge bg-primary rounded-pill">${{
articulo.listaprecio.precio_1 }}</span>

    </li>

    <li class="list-group-item d-flex justify-content-between
align-items-center">

        <span>Precio 2:</span>

        <span class="badge bg-primary rounded-pill">${{
articulo.listaprecio.precio_2 }}</span>

    </li>

    <li class="list-group-item d-flex justify-content-between
align-items-center">

        <span>Precio 3:</span>

        <span class="badge bg-primary rounded-pill">${{
articulo.listaprecio.precio_3 }}</span>

    </li>

    <li class="list-group-item d-flex justify-content-between
align-items-center">

        <span>Precio 4:</span>

        <span class="badge bg-primary rounded-pill">${{
articulo.listaprecio.precio_4 }}</span>

    </li>
</ul>

<hr>

<ul class="list-group list-group-flush">

    <li class="list-group-item d-flex justify-content-between
align-items-center">

        <span><b>Precio de Compra:</b></span>

```

```

        <span class="badge bg-info rounded-pill">${{
articulo.listaprecio.precio_compra }}</span>

    </li>

    <li class="list-group-item d-flex justify-content-between
align-items-center">

        <span><b>Precio de Costo:</b></span>

        <span class="badge bg-info rounded-pill">${{
articulo.listaprecio.precio_costo }}</span>

    </li>

</ul>

</div>

</div>

<!-- Acciones rápidas -->

<div class="card shadow">

    <div class="card-header bg-white">

        <h5 class="mb-0">Acciones Rápidas</h5>

    </div>

    <div class="card-body">

        <div class="d-grid gap-2">

            <button class="btn btn-success">

                <i class="fas fa-plus-circle me-2"></i>Añadir Stock

            </button>

            <button class="btn btn-warning">

                <i class="fas fa-sync-alt me-2"></i>Actualizar Precios

            </button>

        </div>

    </div>

</div>

```

```

        <button class="btn btn-info">
            <i class="fas fa-barcode me-2"></i>Imprimir Código
        </button>

        <button class="btn btn-danger"
onclick="confirmarEliminacion(event, '¿Desea eliminar este artículo?')">
            <i class="fas fa-trash me-2"></i>Eliminar Artículo
        </button>

    </div>

</div>

</div>

</div>

{% endblock %}

```

Paso 12: Crear un formulario para artículos (form.html)

En este paso se crea una template para formularios de creación y edición de artículos.

Elementos clave de Django:

- **{{ form.instance.articulo_id }}**: Accede a un atributo del objeto que se está editando.
- **{{ form.non_field_errors }}**: Muestra errores que no pertenecen a un campo específico.
- **{{ form.campo.errors }}**: Muestra errores específicos de un campo.
- **{% csrf_token %}**: Genera un token CSRF para proteger contra ataques cross-site request forgery.

CSRF significa cross-site request forgery. En este tipo de ataque, el atacante envía un enlace en forma de sms, correo electrónico o chat. De este modo, el atacante engaña al usuario que ya está autenticado en el sitio web para que realice diversas acciones, como transferir fondos,

cambiar el correo electrónico, etcétera. Dependiendo de la naturaleza del ataque, el atacante puede obtener acceso completo a la cuenta.

Django proporciona una característica para prevenir este tipo de ataques maliciosos. Cuando un usuario se autentica y navega por el sitio web, Django genera un token CSRF único para cada sesión. Este token se incluye en los formularios o peticiones enviadas por el usuario y es comprobado por el servidor para verificar que la petición proviene del usuario autenticado y no de una fuente maliciosa.

Python

```
{% extends 'base.html' %}

{% block title %}

    {% if form.instance.articulo_id %}Editar{% else %}Nuevo{% endif %} Artículo
- Sistema POS

{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3">

        <i class="fas fa-{% if form.instance.articulo_id %}edit{% else %}plus{%
endif %} me-2"></i>

        {% if form.instance.articulo_id %}Editar{% else %}Nuevo{% endif %}
Artículo

    </h2>

    <a href="{% url 'articulos_list' %}" class="btn btn-outline-secondary">

        <i class="fas fa-arrow-left me-1"></i>Volver

    </a>

</div>
```



```

<div class="card shadow">

    <div class="card-header bg-white">

        <h5 class="mb-0">Información del Artículo</h5>

    </div>

    <div class="card-body">

        <form method="post" id="articuloForm" class="needs-validation"
novalidate>

            {% csrf_token %}

            {% if form.non_field_errors %}

            <div class="alert alert-danger" role="alert">

                <ul class="mb-0">

                    {% for error in form.non_field_errors %}

                    <li>{{ error }}</li>

                    {% endfor %}

                </ul>

            </div>

            {% endif %}

            <div class="row mb-3">

                <div class="col-md-6">

                    <label for="{{ form.codigo_articulo.id_for_label }}"
class="form-label">Código de Artículo*</label>

                    <input type="text" class="form-control {% if
form.codigo_articulo.errors %}is-invalid{% endif %}"

```

```

        id="{{ form.codigo_articulo.id_for_label }}"

        name="{{ form.codigo_articulo.name }}"

        value="{{ form.codigo_articulo.value|default:'' }}"

        required>

<div class="invalid-feedback">

    {% for error in form.codigo_articulo.errors %}

        {{ error }}

    {% endfor %}

</div>

</div>

<div class="col-md-6">

    <label for="{{ form.codigo_barras.id_for_label }}"
class="form-label">Código de Barras</label>

    <input type="text" class="form-control {% if
form.codigo_barras.errors %}is-invalid{% endif %}"

        id="{{ form.codigo_barras.id_for_label }}"

        name="{{ form.codigo_barras.name }}"

        value="{{ form.codigo_barras.value|default:'' }}">

    <div class="invalid-feedback">

        {% for error in form.codigo_barras.errors %}

            {{ error }}

        {% endfor %}

    </div>

</div>

</div>

```

```

<div class="mb-3">

    <label for="{{ form.descripcion.id_for_label }}"
class="form-label">Descripción</label>

    <input type="text" class="form-control {% if
form.descripcion.errors %}is-invalid{% endif %}"

        id="{{ form.descripcion.id_for_label }}"

        name="{{ form.descripcion.name }}"

        value="{{ form.descripcion.value|default:'' }}"

        required>

    <div class="invalid-feedback">

        {% for error in form.descripcion.errors %}

            {{ error }}

        {% endfor %}

    </div>

</div>


<div class="mb-3">

    <label for="{{ form.presentacion.id_for_label }}"
class="form-label">Presentación</label>

    <input type="text" class="form-control {% if
form.presentacion.errors %}is-invalid{% endif %}"

        id="{{ form.presentacion.id_for_label }}"

        name="{{ form.presentacion.name }}"

        value="{{ form.presentacion.value|default:'' }}">

    <div class="invalid-feedback">

        {% for error in form.presentacion.errors %}

            {{ error }}


```

```

        {% endfor %}

    </div>

</div>

<div class="row mb-3">

    <div class="col-md-6">

        <label for="{{ form.grupo.id_for_label }}"
class="form-label">Grupo*</label>

        <select class="form-select {% if form.grupo.errors
%}is-invalid{% endif %}"

            id="{{ form.grupo.id_for_label }}"

            name="{{ form.grupo.name }}"

            required>

            <option value="">Seleccione un grupo</option>

            {% for choice in form.grupo.field.queryset %}

                <option value="{{ choice.pk }}" {% if
form.grupo.value|stringformat:"s" == choice.pk|stringformat:"s" %}selected{%
endif %}>

                    {{ choice }}

                </option>

            {% endfor %}

        </select>

        <div class="invalid-feedback">

            {% for error in form.grupo.errors %}

                {{ error }}

            {% endfor %}

        </div>

```

```

        </div>

        <div class="col-md-6">

            <label for="{{ form.linea.id_for_label }}"
class="form-label">Línea*</label>

            <select class="form-select {% if form.linea.errors
%}is-invalid{% endif %}"

                id="{{ form.linea.id_for_label }}"

                name="{{ form.linea.name }}"

                required>

                <option value="">Seleccione una línea</option>

                {% for choice in form.linea.field.queryset %}

                    <option value="{{ choice.pk }}" {% if
form.linea.value|stringformat:"s" == choice.pk|stringformat:"s" %}selected{%
endif %}>

                        {{ choice }}

                    </option>

                {% endfor %}

            </select>

            <div class="invalid-feedback">

                {% for error in form.linea.errors %}

                    {{ error }}

                {% endfor %}

            </div>

        </div>

    </div>

    <div class="mb-3">

```

```

        <label for="{{ form.stock.id_for_label }}"
class="form-label">Stock Inicial</label>

        <input type="number" class="form-control {% if
form.stock.errors %}is-invalid{% endif %}"

            id="{{ form.stock.id_for_label }}"

            name="{{ form.stock.name }}"

            value="{{ form.stock.value|default:'0' }}"

            step="0.01">

        <div class="invalid-feedback">

            {% for error in form.stock.errors %}

                {{ error }}

            {% endfor %}

        </div>
    </div>

<hr class="my-4">

<h5>Precios</h5>

<div class="row mb-3">

    <div class="col-md-6">

        <label for="{{ precio_form.precio_1.id_for_label }}"
class="form-label">Precio 1*</label>

        <div class="input-group">

            <span class="input-group-text">$</span>

            <input type="number" class="form-control {% if
precio_form.precio_1.errors %}is-invalid{% endif %}"

                id="{{ precio_form.precio_1.id_for_label }}"

```

```

        name="{{ precio_form.precio_1.name }}"
        value="{{ precio_form.precio_1.value|default:'0'
    }}"

        step="0.01" required>

<div class="invalid-feedback">

    {% for error in precio_form.precio_1.errors %}

        {{ error }}

    {% endfor %}

</div>

</div>

<div class="col-md-6">

    <label for="{{ precio_form.precio_2.id_for_label }}"
class="form-label">Precio 2</label>

    <div class="input-group">

        <span class="input-group-text">$</span>

        <input type="number" class="form-control {% if
precio_form.precio_2.errors %}is-invalid{% endif %}"

            id="{{ precio_form.precio_2.id_for_label }}"

            name="{{ precio_form.precio_2.name }}"

            value="{{ precio_form.precio_2.value|default:'0'
    }}"

            step="0.01">

        <div class="invalid-feedback">

            {% for error in precio_form.precio_2.errors %}

                {{ error }}

            {% endfor %}


```

```

        </div>

    </div>

</div>

</div>

<div class="row mb-3">

    <div class="col-md-6">

        <label for="{{ precio_form.precio_3.id_for_label }}"
class="form-label">Precio 3</label>

        <div class="input-group">

            <span class="input-group-text">${</span>

            <input type="number" class="form-control {% if
precio_form.precio_3.errors %}is-invalid{% endif %}"

                id="{{ precio_form.precio_3.id_for_label }}"
                name="{{ precio_form.precio_3.name }}"
                value="{{ precio_form.precio_3.value|default:'0'
}}}"

                step="0.01">

            <div class="invalid-feedback">

                {% for error in precio_form.precio_3.errors %}

                    {{ error }}

                {% endfor %}

            </div>

        </div>

    </div>

</div>

<div class="col-md-6">

```



```

        <label for="{{ precio_form.precio_4.id_for_label }}"
class="form-label">Precio 4</label>

        <div class="input-group">

            <span class="input-group-text">$</span>

            <input type="number" class="form-control {% if
precio_form.precio_4.errors %}is-invalid{% endif %}"

                id="{{ precio_form.precio_4.id_for_label }}"

                name="{{ precio_form.precio_4.name }}"

                value="{{ precio_form.precio_4.value|default:'0'
}}"

                step="0.01">

            <div class="invalid-feedback">

                {% for error in precio_form.precio_4.errors %}

                    {{ error }}

                {% endfor %}

            </div>

        </div>

    </div>

</div>

<div class="row mb-3">

    <div class="col-md-6">

        <label for="{{ precio_form.precio_compra.id_for_label }}"
class="form-label">Precio de Compra</label>

        <div class="input-group">

            <span class="input-group-text">$</span>

```

```

        <input type="number" class="form-control {% if
precio_form.precio_compra.errors %}is-invalid{% endif %}"

        id="{{ precio_form.precio_compra.id_for_label
}}"

        name="{{ precio_form.precio_compra.name }}"

        value="{{
precio_form.precio_compra.value|default:'0' }}"

        step="0.01" required>

        <div class="invalid-feedback">

            {% for error in precio_form.precio_compra.errors %}

                {{ error }}

            {% endfor %}

        </div>

    </div>

</div>

<div class="col-md-6">

    <label for="{{ precio_form.precio_costo.id_for_label }}"
class="form-label">Precio de Costo*</label>

    <div class="input-group">

        <span class="input-group-text">$</span>

        <input type="number" class="form-control {% if
precio_form.precio_costo.errors %}is-invalid{% endif %}"

        id="{{ precio_form.precio_costo.id_for_label }}"

        name="{{ precio_form.precio_costo.name }}"

        value="{{
precio_form.precio_costo.value|default:'0' }}"

        step="0.01" required>

        <div class="invalid-feedback">

```

```

        {% for error in precio_form.precio_costo.errors %}
            {{ error }}
        {% endfor %}
    </div>
</div>
</div>
</div>

<div class="d-grid gap-2 d-md-flex justify-content-md-end mt-4">
    <button type="button" class="btn btn-outline-secondary me-md-2"
onclick="history.back()">Cancelar</button>

    <button type="submit" class="btn btn-primary">Guardar
Artículo</button>
</div>
</form>
</div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    document.addEventListener('DOMContentLoaded', function () {
        // Validación del formulario
        validarFormulario('articuloForm');

        // Cargar líneas según el grupo seleccionado
    }

```

```

const grupoSelect = document.getElementById('{{ form.grupo.id_for_label
}}');

const lineaSelect = document.getElementById('{{ form.linea.id_for_label
}}');

if (grupoSelect && lineaSelect) {
    grupoSelect.addEventListener('change', function() {
        // Aquí implementarías la lógica para cargar líneas según el
        grupo

        // Normalmente esto se haría mediante una petición AJAX al
        servidor

        // Ejemplo simplificado:
        const groupId = this.value;
        if (groupId) {
            // Código de ejemplo - en un caso real, harías una petición
            AJAX

            // fetch(`/api/lineas-por-grupo/${groupId}/`)
            //     .then(resp => resp.json())
            //     .then(data => {
            //         // Llenar el select de líneas
            //     });
        }
    });
}

});
</script>
{% endblock %}

```

Paso 13: Crear una vista para login (login.html)

Para este paso vamos a crear nuestra página para el login, para ello vamos a ir al archivo `templates/accounts/login.html` y vamos a codificar lo siguiente:

Python

```
{% extends 'base.html' %}

{% load static %}

{% block title %}Iniciar Sesión - Sistema POS{% endblock %}

{% block content %}

<div class="row justify-content-center mt-5">

    <div class="col-md-6 col-lg-5">

        <div class="text-center mb-4">

            <h2 class="mt-3">Sistema POS</h2>

            <p class="text-muted">Ingrese sus credenciales para acceder</p>

        </div>

        <div class="card shadow">

            <div class="card-body p-4">

                {% if form.errors %}

                    <div class="alert alert-danger" role="alert">

                        <i class="fas fa-exclamation-triangle me-2"></i>Nombre de
                        usuario o contraseña incorrectos

                    </div>

                {% endif %}

            </div>

        </div>

    </div>

</div>
```

```

{% endif %}

<form method="post" action="{% url 'login' %}">

    {% csrf_token %}

    <div class="mb-3">

        <label for="id_username" class="form-label">Nombre de
usuario</label>

        <div class="input-group">

            <span class="input-group-text"><i class="fas
fa-user"></i></span>

            <input type="text" name="username" id="id_username"
class="form-control" placeholder="Ingrese su usuario" required autofocus>

        </div>

    </div>

    <div class="mb-4">

        <label for="id_password"
class="form-label">Contraseña</label>

        <div class="input-group">

            <span class="input-group-text"><i class="fas
fa-lock"></i></span>

            <input type="password" name="password"
id="id_password" class="form-control" placeholder="Ingrese su contraseña"
required>

        </div>

    </div>

```

```

        <div class="form-check mb-3">
            <input class="form-check-input" type="checkbox"
id="remember-me">
            <label class="form-check-label" for="remember-me">
                Recordarme
            </label>
        </div>

        <div class="d-grid mb-3">
            <button type="submit" class="btn btn-primary btn-lg">
                <i class="fas fa-sign-in-alt me-2"></i>Iniciar
                Sesi3n
            </button>
        </div>

        <input type="hidden" name="next" value="{{ next }}">
    </form>

    <div class="text-center">
        <a href="#" class="text-decoration-none">¿Olvid3 su
contrase~a?</a>
    </div>
</div>

<div class="text-center mt-3 text-muted">

```

```

        <small>&copy; { % now "Y" %} Sistema POS. Todos los derechos
reservados.</small>

    </div>

</div>
</div>
{% endblock %}

{% block extra_js %}
<script>
    document.addEventListener('DOMContentLoaded', function() {
        // Animación simple para la tarjeta de login
        const loginCard = document.querySelector('.card');
        if (loginCard) {
            loginCard.style.opacity = '0';
            loginCard.style.transform = 'translateY(20px)';
            loginCard.style.transition = 'opacity 0.5s ease, transform 0.5s
ease';

            setTimeout(function() {
                loginCard.style.opacity = '1';
                loginCard.style.transform = 'translateY(0)';
            }, 100);
        }
    });
</script>
{% endblock %}

```


Paso 13: Crear una vista para el perfil de usuario (profile.html)

Para este paso vamos a crear nuestra página para poder ingresar la información o perfil de usuario, para ello vamos a ir al archivo `templates/accounts/profile.html` y vamos a codificar lo siguiente:

Python

```
{% extends 'base.html' %}

{% load static %}

{% block title %}Mi Perfil - Sistema POS{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3"><i class="fas fa-user-circle me-2"></i>Mi Perfil</h2>

    <a href="{% url 'home' %}" class="btn btn-outline-secondary">

        <i class="fas fa-arrow-left me-1"></i>Volver

    </a>

</div>

<div class="row">

    <div class="col-md-4">

        <div class="card shadow mb-4">

            <div class="card-body text-center">

                <div class="avatar-circle mb-3 mx-auto">

                    <span class="initials">{{ user.full_name|slice:"":1"

                }}</span>

            </div>

        </div>

    </div>

</div>
```

```

<h4>{{ user.full_name }}</h4>

<p class="text-muted">{{ user.perfil.perfil_nombre }}</p>

<hr>

<div class="d-grid">

    <button class="btn btn-outline-primary">

        <i class="fas fa-camera me-2"></i>Cambiar Foto

    </button>

</div>

</div>

</div>

<div class="card shadow">

    <div class="card-header bg-white">

        <h5 class="mb-0">Acceso Rápido</h5>

    </div>

    <div class="card-body p-0">

        <div class="list-group list-group-flush">

            <a href="#" class="list-group-item list-group-item-action">

                <i class="fas fa-key me-2"></i>Cambiar Contraseña

            </a>

            <a href="#" class="list-group-item list-group-item-action">

                <i class="fas fa-history me-2"></i>Historial de

Actividades

            </a>

            <a href="#" class="list-group-item list-group-item-action">

                <i class="fas fa-cog me-2"></i>Preferencias

```

```

        </a>

        <a href="#" class="list-group-item list-group-item-action
text-danger">

            <i class="fas fa-sign-out-alt me-2"></i>Cerrar Sesión

        </a>

    </div>

</div>

</div>

</div>

<div class="col-md-8">

    <div class="card shadow">

        <div class="card-header bg-white d-flex justify-content-between
align-items-center">

            <h5 class="mb-0">Información Personal</h5>

            <button class="btn btn-sm btn-primary edit-profile-btn">

                <i class="fas fa-edit me-1"></i>Editar

            </button>

        </div>

        <div class="card-body">

            <div class="view-mode">

                <div class="row mb-3">

                    <div class="col-md-6">

                        <p class="mb-1 text-muted">Nombre Completo:</p>

                        <h5>{{ user.full_name }}</h5>

                    </div>

                </div>

            </div>

        </div>

    </div>

</div>

```

```

<div class="col-md-6">
    <p class="mb-1 text-muted">Nombre de Usuario:</p>
    <h5>{{ user.username }}</h5>
</div>
</div>

<div class="row mb-3">
    <div class="col-md-6">
        <p class="mb-1 text-muted">Correo Electrónico:</p>
        <h5>{{ user.email }}</h5>
    </div>
    <div class="col-md-6">
        <p class="mb-1 text-muted">Teléfono Móvil:</p>
        <h5>{{ user.mobile }}</h5>
    </div>
</div>

<div class="row mb-3">
    <div class="col-md-6">
        <p class="mb-1 text-muted">Perfil:</p>
        <h5>{{ user.perfil.perfil_nombre }}</h5>
    </div>
    <div class="col-md-6">
        <p class="mb-1 text-muted">Último Acceso:</p>
        <h5>{{ user.last_login|date:'d/m/Y H:i' }}</h5>
    </div>
</div>

```

```

        </div>

    </div>

</div>

<div class="edit-mode" style="display: none;">
    <form method="post" action="{% url 'profile_update' %}">
        {% csrf_token %}

        <div class="row mb-3">
            <div class="col-md-6">
                <label for="full_name"
class="form-label">Nombre Completo*</label>

                <input type="text" class="form-control"
id="full_name" name="full_name" value="{{ user.full_name }}" required>

            </div>

            <div class="col-md-6">

                <label for="username" class="form-label">Nombre
de Usuario*</label>

                <input type="text" class="form-control"
id="username" name="username" value="{{ user.username }}" readonly>

                <div class="form-text">El nombre de usuario no
puede ser modificado.</div>

            </div>

        </div>

        <div class="row mb-3">
            <div class="col-md-6">

```

```

        <label for="email" class="form-label">Correo
Electrónico*</label>

        <input type="email" class="form-control"
id="email" name="email" value="{{ user.email }}" required>

    </div>

    <div class="col-md-6">

        <label for="mobile" class="form-label">Teléfono
Móvil*</label>

        <input type="tel" class="form-control"
id="mobile" name="mobile" value="{{ user.mobile }}" required>

    </div>

</div>

<div class="d-flex justify-content-end mt-4">

    <button type="button" class="btn
btn-outline-secondary me-2 cancel-edit-btn">Cancelar</button>

    <button type="submit" class="btn
btn-primary">Guardar Cambios</button>

</div>

</form>

</div>

</div>

</div>

<div class="card shadow mt-4">

    <div class="card-header bg-white">

        <h5 class="mb-0">Actividad Reciente</h5>

    </div>

```

```

<div class="card-body p-0">
  <div class="timeline">
    <div class="timeline-item">
      <div class="timeline-marker bg-success"></div>
      <div class="timeline-content">
        <h6 class="mb-0">Inicio de sesión</h6>
        <p class="text-muted mb-0 small">Hoy, 10:45 AM</p>
        <p>Inició sesión desde 192.168.1.24</p>
      </div>
    </div>
    <div class="timeline-item">
      <div class="timeline-marker bg-primary"></div>
      <div class="timeline-content">
        <h6 class="mb-0">Creación de artículo</h6>
        <p class="text-muted mb-0 small">Ayer, 15:30 PM</p>
        <p>Creó el artículo "Camisa Algodón Talla M"</p>
      </div>
    </div>
    <div class="timeline-item">
      <div class="timeline-marker bg-warning"></div>
      <div class="timeline-content">
        <h6 class="mb-0">Modificación de precios</h6>
        <p class="text-muted mb-0 small">24/04/2025, 09:15
AM</p>
        <p>Actualizó los precios de 5 artículos</p>
      </div>
    </div>
  </div>

```

```

        </div>

    </div>

</div>

<div class="card-footer bg-white text-center">

    <a href="#" class="btn btn-sm btn-outline-primary">Ver Todo el
Historial</a>

</div>

</div>

</div>

</div>

{% endblock %}

{% block extra_css %}

<style>

    /* Estilos para el avatar */

    .avatar-circle {

        width: 100px;

        height: 100px;

        background-color: #007bff;

        border-radius: 50%;

        display: flex;

        align-items: center;

        justify-content: center;

    }

    .initials {

```



```
        font-size: 40px;
        color: white;
        font-weight: bold;
    }

    /* Estilos para la línea de tiempo */
    .timeline {
        position: relative;
        padding-left: 30px;
    }

    .timeline::before {
        content: '';
        position: absolute;
        top: 0;
        bottom: 0;
        left: 15px;
        width: 2px;
        background-color: #e9ecef;
    }

    .timeline-item {
        position: relative;
        padding-bottom: 20px;
    }
```

```
.timeline-marker {
  position: absolute;
  top: 5px;
  left: -30px;
  width: 12px;
  height: 12px;
  border-radius: 50%;
}

.timeline-content {
  padding-top: 3px;
}
</style>

{% endblock %}

{% block extra_js %}
<script>
  document.addEventListener('DOMContentLoaded', function() {

    // Manejar la edición del perfil

    const editBtn = document.querySelector('.edit-profile-btn');
    const cancelBtn = document.querySelector('.cancel-edit-btn');
    const viewMode = document.querySelector('.view-mode');
    const editMode = document.querySelector('.edit-mode');
```

```

    if (editBtn && cancelBtn && viewMode && editMode) {
        editBtn.addEventListener('click', function() {
            viewMode.style.display = 'none';
            editMode.style.display = 'block';
            this.style.display = 'none';
        });

        cancelBtn.addEventListener('click', function() {
            viewMode.style.display = 'block';
            editMode.style.display = 'none';
            editBtn.style.display = 'inline-block';
        });
    }
});
</script>
{% endblock %}

```

Paso 14: Creando Vistas en Django (core/views.py)

Una View es una página web que tiene una funcionalidad (backend) y una cara visible (frontend/templates). Es una estructura de Python que puede ser una función o una clase.

- Cuando está construida mediante funciones nos referimos al concepto de Function Based Views
- Cuando está construida mediante clases nos referimos al concepto de Generic Views

Vistas basadas en funciones

Como su nombre indica, las vistas basadas en funciones se implementan como funciones de Python. Para entender cómo funcionan, considere el siguiente fragmento, que muestra una función de vista simple llamada `home_page`:

```
from django.http import HttpResponse
def home_page(request):
    message = "<html><h1>Welcome to my Website</h1></html>"
    return HttpResponse(message)
```

La función de vista definida aquí, llamada `home_page`, toma un objeto `request` como argumento y devuelve un objeto `HttpResponse` con un mensaje `Welcome to my Website`. La ventaja de utilizar vistas basadas en funciones es que, al estar implementadas como simples funciones de Python, son más fáciles de aprender y también de leer para otros programadores. La mayor desventaja de las vistas basadas en funciones es que el código no puede reutilizarse y hacerse tan conciso como las vistas basadas en clases para casos de uso genérico. La siguiente sección es una breve introducción a las vistas basadas en clases.

Vistas basadas en clases

Como su nombre indica, las vistas basadas en clases se implementan como clases de Python. Utilizando los principios de la herencia de clases, estas clases se implementan como subclases de las clases de vistas genéricas de Django. A diferencia de las vistas basadas en funciones, donde toda la lógica de la vista se expresa explícitamente en una función, las clases de vistas genéricas de Django vienen con varias propiedades y métodos pre-construidos que pueden proporcionar atajos para escribir vistas limpias y reutilizables. Esta propiedad resulta útil con bastante frecuencia durante el desarrollo web; por ejemplo, los desarrolladores a menudo necesitan renderizar una página HTML sin necesidad de insertar ningún dato de la base de datos, o cualquier personalización específica para el usuario. En este caso, es posible simplemente heredar de `TemplateView` de Django, y especificar la ruta del archivo HTML. El siguiente es un ejemplo de una vista basada en clases que puede mostrar el mismo mensaje que en el ejemplo de vista basada en funciones:

```
from django.views.generic import TemplateView
class HomePage(TemplateView):
    template_name = 'home_page.html'
```

En el fragmento de código anterior, HomePage es una vista basada en clases que hereda TemplateView de Django del módulo django.views.generic. El atributo de clase template_name define la plantilla a mostrar cuando la vista es invocada.

Para este laboratorio, vamos a usar vistas basadas en funciones y ahora vamos a definir algunas vistas en el archivo `core/views.py`:

Python

```
from django.shortcuts import render, redirect, get_object_or_404
from django.core.paginator import Paginator
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import JsonResponse
import uuid

from .models import Artículo, GrupoArticulo, LineaArticulo, ListaPrecio
from .forms import ArtículoForm, ListaPrecioForm # Necesitarás crear estos
forms

@login_required
def home(request):
    """Vista para la página principal"""
    # Obtener datos para los widgets
    total_articulos = Artículo.objects.count()
    total_usuarios = 0 # Obtener de tu modelo Usuario
    bajo_stock = Artículo.objects.filter(stock__lt=10).count()

    context = {
```

```

        'total_articulos': total_articulos,
        'total_usuarios': total_usuarios,
        'bajo_stock': bajo_stock,
        'ventas_hoy': 0, # En un caso real, calcularías las ventas del día
    }

    return render(request, 'core/index.html', context)

```

@login_required

```

def articulos_list(request):
    """Vista para listar artículos"""

    articulos_list = Articulo.objects.all()

    # Filtros (podrías expandir esto)

    q = request.GET.get('q')

    if q:
        articulos_list = articulos_list.filter(descripcion__icontains=q)

    # Paginación

    paginator = Paginator(articulos_list, 15) # 15 artículos por página

    page_number = request.GET.get('page')

    articulos = paginator.get_page(page_number)

    context = {
        'articulos': articulos,
    }

```

```

        return render(request, 'core/articulos/list.html', context)

@login_required
def articulo_detail(request, articulo_id):
    """Vista para ver el detalle de un artículo"""
    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)

    context = {
        'articulo': articulo,
    }

    return render(request, 'core/articulos/detail.html', context)

@login_required
def articulo_create(request):
    """Vista para crear un nuevo artículo"""
    if request.method == 'POST':
        form = ArticuloForm(request.POST)
        precio_form = ListaPrecioForm(request.POST)

        if form.is_valid() and precio_form.is_valid():
            # Generar ID para el artículo
            articulo = form.save(commit=False)
            articulo.articulo_id = uuid.uuid4()
            articulo.save()

```

```

        # Guardar precios

        lista_precio = precio_form.save(commit=False)

        lista_precio.articulo = articulo

        lista_precio.save()

        messages.success(request, 'Artículo creado correctamente.')

        return redirect('articulo_detail',
articulo_id=articulo.articulo_id)

    else:

        form = ArticuloForm()

        precio_form = ListaPrecioForm()

context = {

    'form': form,

    'precio_form': precio_form,

}

    return render(request, 'core/articulos/form.html', context)

@login_required
def articulo_edit(request, articulo_id):

    """Vista para editar un artículo existente"""

    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)

    lista_precio = get_object_or_404(ListaPrecio, articulo=articulo)

    if request.method == 'POST':

        form = ArticuloForm(request.POST, instance=articulo)

```



```

precio_form = ListaPrecioForm(request.POST, instance=lista_precio)

if form.is_valid() and precio_form.is_valid():
    form.save()
    precio_form.save()

    messages.success(request, 'Artículo actualizado correctamente.')
    return redirect('articulo_detail',
articulo_id=articulo.articulo_id)
else:
    form = ArticuloForm(instance=articulo)
    precio_form = ListaPrecioForm(instance=lista_precio)

context = {
    'form': form,
    'precio_form': precio_form,
}

return render(request, 'core/articulos/form.html', context)

@login_required
def articulo_delete(request, articulo_id):
    """Vista para eliminar un artículo"""
    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)

    if request.method == 'POST':
        articulo.delete()

```

```

        messages.success(request, 'Artículo eliminado correctamente.')

        return redirect('articulos_list')

context = {

    'articulo': articulo,

}

return render(request, 'core/articulos/delete.html', context)

# Vistas para obtener información por AJAX

@login_required

def get_lineas_por_grupo(request, grupo_id):

    """API para obtener líneas de artículo según el grupo seleccionado"""

    lineas = LineaArticulo.objects.filter(grupo_id=grupo_id, estado=1)

    data = [{'id': str(linea.linea_id), 'nombre': linea.nombre_linea} for linea
in lineas]

    return JsonResponse(data, safe=False)

```

Elementos clave de Django:

- Decorador `@login_required`: Asegura que solo usuarios autenticados puedan acceder a una vista.
- `Paginator`: Clase para implementar paginación de resultados.
- `get_object_or_404`: Función útil para obtener un objeto o devolver un 404 si no existe.
- `messages`: Framework para mostrar mensajes flash a los usuarios.

Paso 15: Crear los formularios para artículos en `core/forms.py`

Cuando trabajamos con una aplicación web interactiva, no sólo queremos proporcionar datos a los usuarios, sino también aceptar datos de ellos, ya sea para personalizar las respuestas que estamos generando o para permitirles enviar datos al sitio. Al navegar por Internet, seguro que has utilizado formularios. Tanto si estás conectado a tu cuenta de banca por Internet, navegando por la web con un navegador, publicando un mensaje en las redes sociales o escribiendo un correo electrónico a un cliente de correo electrónico en línea, en todos estos casos, estás introduciendo datos en un formulario. Un formulario se compone de entradas que definen pares clave-valor de datos para enviar al servidor. Por ejemplo, al iniciar sesión en un sitio web, los datos que se envían tendrían las claves nombre de usuario y contraseña, con los valores de su nombre de usuario y su contraseña, respectivamente. Hablaremos de los diferentes tipos de entradas con más detalle en una próxima sección. Cada entrada del formulario tiene un nombre, y así es como se identifican sus datos en el lado del servidor (en una vista de Django). Puede haber múltiples entradas con el mismo nombre, cuyos datos están disponibles en una lista que contiene todos los valores publicados con este nombre - por ejemplo, una lista de casillas de verificación con permisos para aplicar a los usuarios. Cada casilla de verificación tendría el mismo nombre pero un valor diferente. El formulario tiene atributos que especifican a qué URL debe enviar los datos el navegador y qué método debe utilizar para enviarlos (los navegadores sólo admiten GET o POST).

Creación de formularios en Django

Puedes crear formularios en Django manualmente con la clase `Form` o generados automáticamente a partir de modelos usando `ModelForm`.

Para crear un formulario en Django, define una clase `Form` en tu archivo `forms.py`. Esta clase especifica los campos del formulario y las reglas de validación para procesar eficazmente la entrada del usuario.

Ahora vamos a crear nuestros formularios para los modelos `Articulo` y `ListaPrecios` para ello vamos al archivo `core/forms.py` (si el archivo no existe hay que crearlo manualmente)

Python

```
from django import forms

from .models import Articulo, ListaPrecio, GrupoArticulo, LineaArticulo
```

```

from pos_project.choices import EstadoEntidades

class ArticuloForm(forms.ModelForm):

    """Formulario para el modelo Articulo"""

    class Meta:

        model = Articulo

        fields = ['codigo_articulo', 'codigo_barras', 'descripcion',
'presentacion',

                'grupo', 'linea', 'stock']

        widgets = {

            'codigo_articulo': forms.TextInput(attrs={'class':
'form-control'}),

            'codigo_barras': forms.TextInput(attrs={'class': 'form-control'}),

            'descripcion': forms.TextInput(attrs={'class': 'form-control'}),

            'presentacion': forms.TextInput(attrs={'class': 'form-control'}),

            'grupo': forms.Select(attrs={'class': 'form-select'}),

            'linea': forms.Select(attrs={'class': 'form-select'}),

            'stock': forms.NumberInput(attrs={'class': 'form-control', 'step':
'0.01'}),

        }

    def __init__(self, *args, **kwargs):

        super().__init__(*args, **kwargs)

        # Filtrar solo grupos activos

        self.fields['grupo'].queryset =
GrupoArticulo.objects.filter(estado=EstadoEntidades.ACTIVO)

```

```

        # Filtrar líneas activas

        self.fields['linea'].queryset =
LineaArticulo.objects.filter(estado=EstadoEntidades.ACTIVO)


        # Si ya existe un grupo seleccionado, filtrar líneas por ese grupo
        if self.instance.pk and self.instance.grupo:

            self.fields['linea'].queryset = LineaArticulo.objects.filter(

                grupo=self.instance.grupo,

                estado=EstadoEntidades.ACTIVO

            )


class ListaPrecioForm(forms.ModelForm):

    """Formulario para el modelo ListaPrecio"""

    class Meta:

        model = ListaPrecio

        fields = ['precio_1', 'precio_2', 'precio_3', 'precio_4',

                'precio_compra', 'precio_costo']

        widgets = {

            'precio_1': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

            'precio_2': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

            'precio_3': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

```

```
        'precio_4': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

        'precio_compra': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

        'precio_costo': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01', 'min': '0'}),

    }
```

En nuestro caso, hemos creado nuestros formularios directamente desde ambos modelos usando `ModelForm`, esto nos permite ahorrar tiempo al generar automáticamente campos de formulario basados en los campos del modelo.

Renderizando formularios en plantillas Django

Para mostrar un formulario en una página web, necesitas renderizarlo en una plantilla. Django proporciona dos enfoques: usar los ayudantes de renderizado incorporados o renderizar el formulario manualmente con HTML.

Renderizado simple con funciones de ayuda

Las funciones de ayuda incorporadas en Django facilitan la personalización de cómo se muestran los formularios en las plantillas. Hay tres métodos para mostrar un formulario con diferentes estructuras:

- `form.as_p` - envuelve cada campo del formulario en una etiqueta `<p>`, ideal para un estilo rápido y limpio.
- `form.as_table` - muestra los campos del formulario en una tabla HTML, útil para diseños estructurados.
- `form.as_ul` - envuelve cada campo en etiquetas `<li>` para un formato tipo lista.

Paso 16: Configurar las vistas en accounts/views.py

En este paso vamos a crear las vistas para nuestra aplicación accounts, para ello vamos a ir al archivo `accounts/views.py` y escribimos el siguiente código:

Python

```
from django.shortcuts import render, redirect

from django.contrib.auth.decorators import login_required
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages

from django.contrib.auth.forms import AuthenticationForm

from .models import Usuario, Perfil

@login_required
def profile_view(request):
    """Vista para mostrar el perfil del usuario"""
    context = {
        'user': request.user,
    }
    return render(request, 'accounts/profile.html', context)

@login_required
def profile_update(request):
    """Vista para actualizar el perfil del usuario"""
    if request.method == 'POST':
```

```
user = request.user

# Actualizar datos del usuario
user.full_name = request.POST.get('full_name')
user.email = request.POST.get('email')
user.mobile = request.POST.get('mobile')
user.save()

messages.success(request, 'Perfil actualizado correctamente.')
return redirect('profile')

return redirect('profile')

def login_view(request):
    """Vista para el login de usuarios"""
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
```



```

        if user is not None:
            login(request, user)
            messages.success(request, f'Bienvenido, {user.full_name}!')

            # Redirigir a la página solicitada o a home
            next_page = request.GET.get('next')
            if next_page:
                return redirect(next_page)
            return redirect('home')
        else:
            messages.error(request, 'Usuario o contraseña incorrectos.')
    else:
        messages.error(request, 'Usuario o contraseña incorrectos.')
else:
    form = AuthenticationForm()

    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)

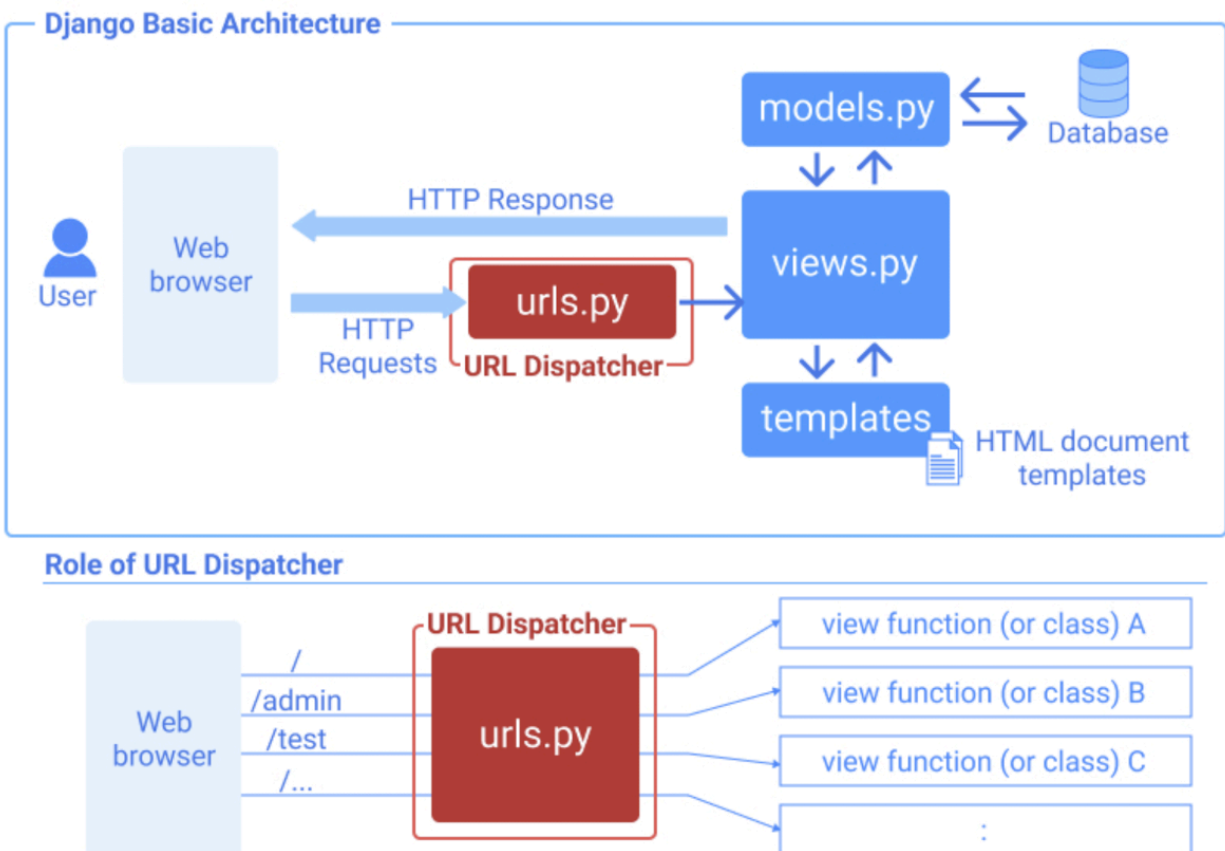
def logout_view(request):
    """Vista para cerrar sesión"""
    logout(request)
    messages.info(request, 'Has cerrado sesión correctamente.')

```

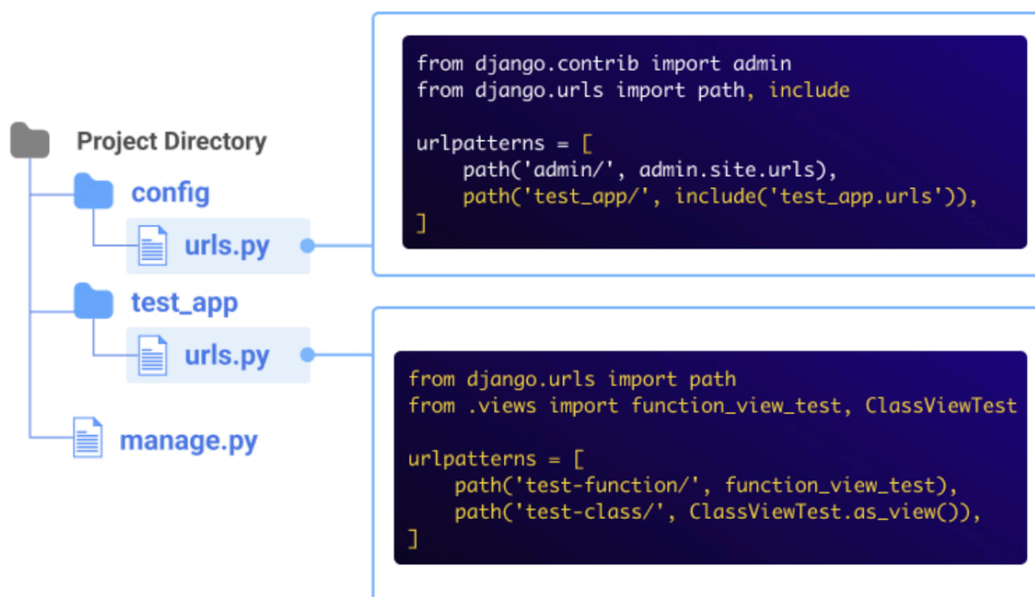
```
return redirect('login')
```

Paso 17: Crear las URLs para nuestras aplicaciones

Django utiliza un sistema basado en patrones para definir las rutas de las aplicaciones web. Estas rutas se configuran en archivos llamados `urls.py` y permiten enlazar las URLs con las vistas.



En Django, las rutas se definen en el archivo `urls.py`. La función principal utilizada es `path`, y opcionalmente, `re_path` para expresiones regulares. Cada aplicación debe definir sus rutas,



Para nuestro caso vamos a crear el archivo de rutas en `core/urls.py` y escribimos lo siguiente:

Python

```
from django.urls import path

from . import views

urlpatterns = [

    # URLs para Artículos

    path('articulos/', views.articulos_list, name='articulos_list'),

    path('articulos/nuevo/', views.articulo_create, name='articulo_create'),

    path('articulos/<uuid:articulo_id>/', views.articulo_detail,
name='articulo_detail'),

    path('articulos/<uuid:articulo_id>/editar/', views.articulo_edit,
name='articulo_edit'),
```

```

    path('articulos/<uuid:articulo_id>/eliminar/', views.articulo_delete,
name='articulo_delete'),

    # API para operaciones AJAX

    path('api/lineas-por-grupo/<uuid:grupo_id>/', views.get_lineas_por_grupo,
name='get_lineas_por_grupo'),

]

```

Ahora vamos a crear nuestras rutas para `accounts/urls.py`:

```

Python

from django.urls import path

from . import views

urlpatterns = [

    path('perfil/', views.profile_view, name='profile'),

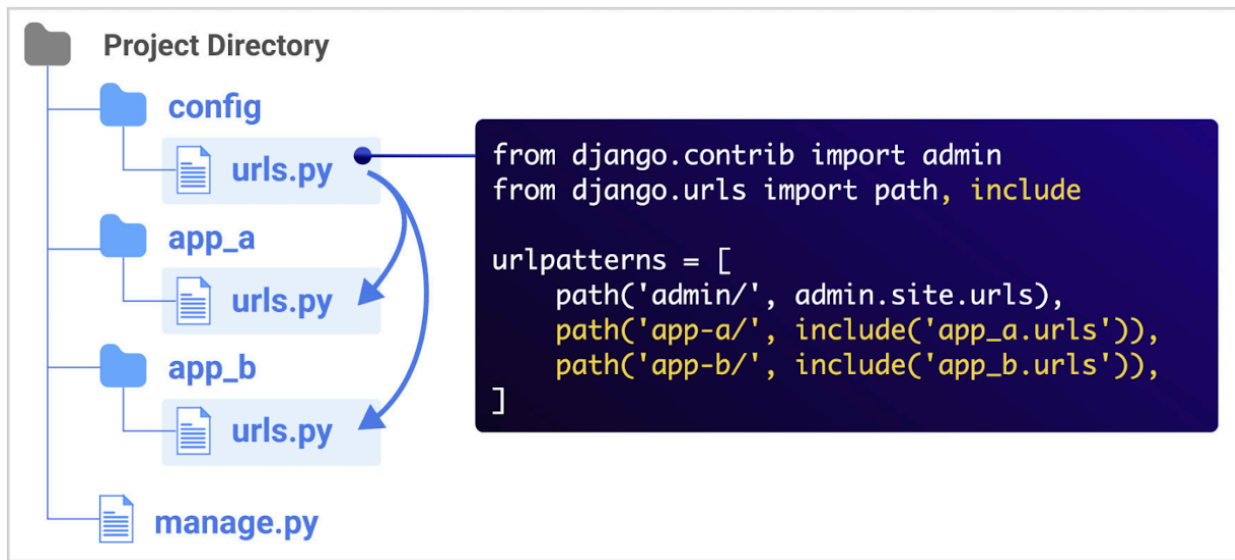
    path('perfil/actualizar/', views.profile_update, name='profile_update'),

]

```

Paso 18: Crear las URLs para nuestro proyecto

Ahora tenemos que configurar a nivel global las urls, que serán usadas en todo nuestro proyecto, para ello tenemos que ir al archivo `pos_project/urls.py` (enrutador principal), desde este archivo se dispararán todas las peticiones http que ingresen a nuestro website dependiendo del tipo de recurso al que se quiera acceder.



Python

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static
from django.contrib.auth.decorators import login_required

from core.views import home
from accounts.views import login_view, logout_view

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', login_required(home), name='home'),
```

```
# URLs de autenticación

path('login/', login_view, name='login'),
path('logout/', logout_view, name='logout'),

# Incluir URLs de apps

path('accounts/', include('accounts.urls')),
path('core/', include('core.urls')),
]

if settings.DEBUG:

    urlpatterns += static(settings.STATIC_URL,
document_root=settings.STATIC_ROOT)

    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```